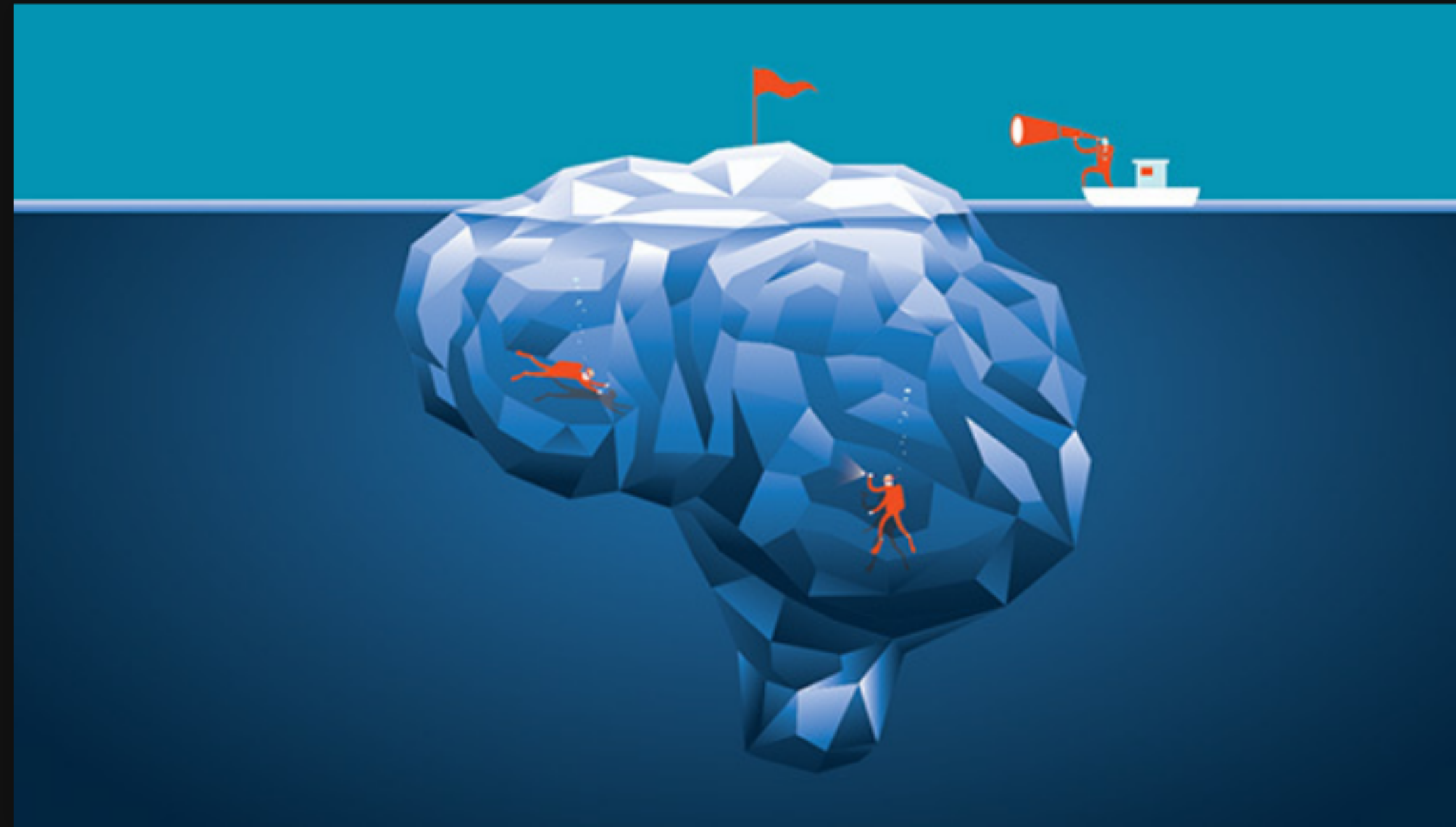


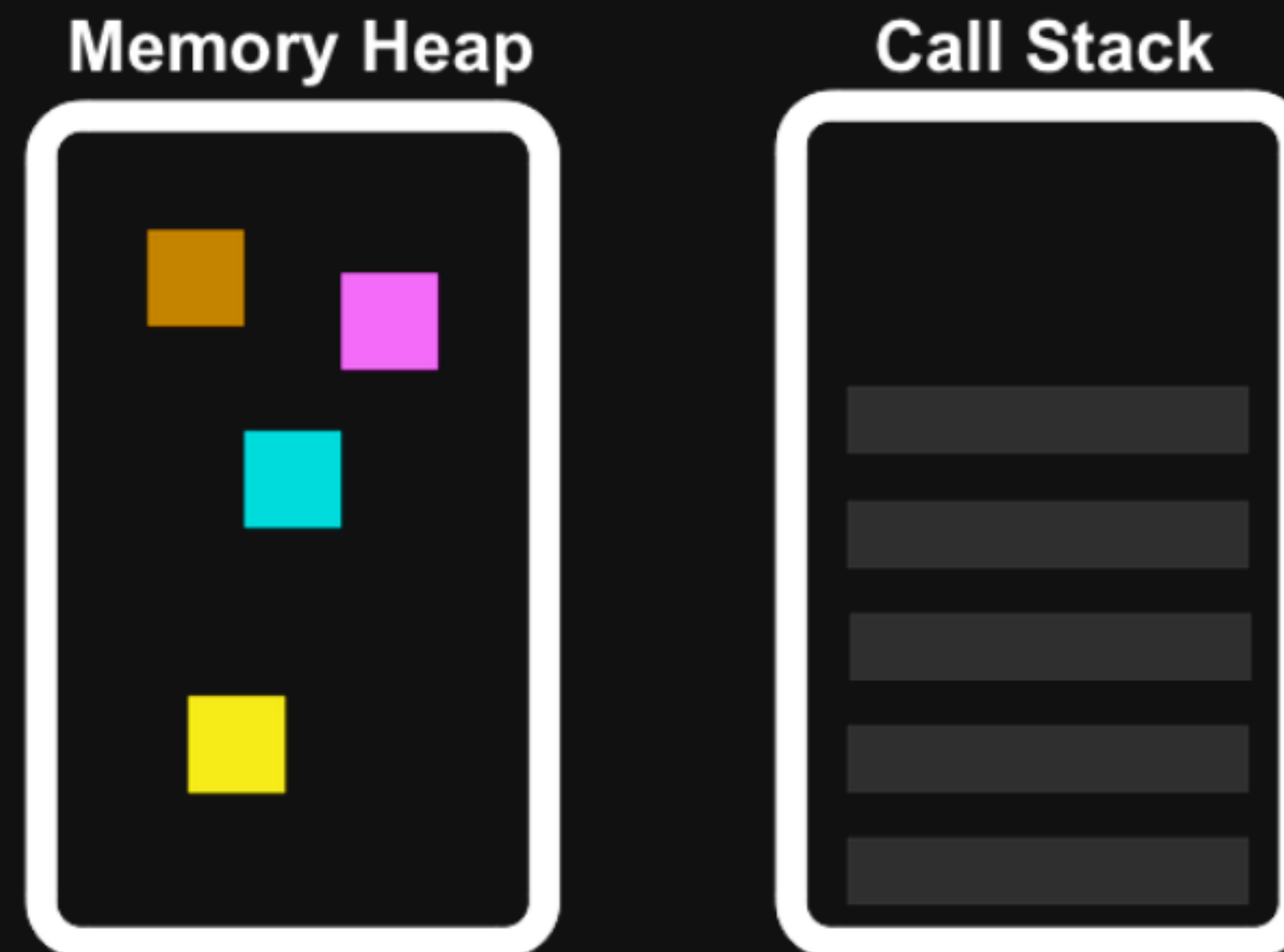


JavaScript Deep Dive

JavaScript Engines



Runtime engines consist of two main components:



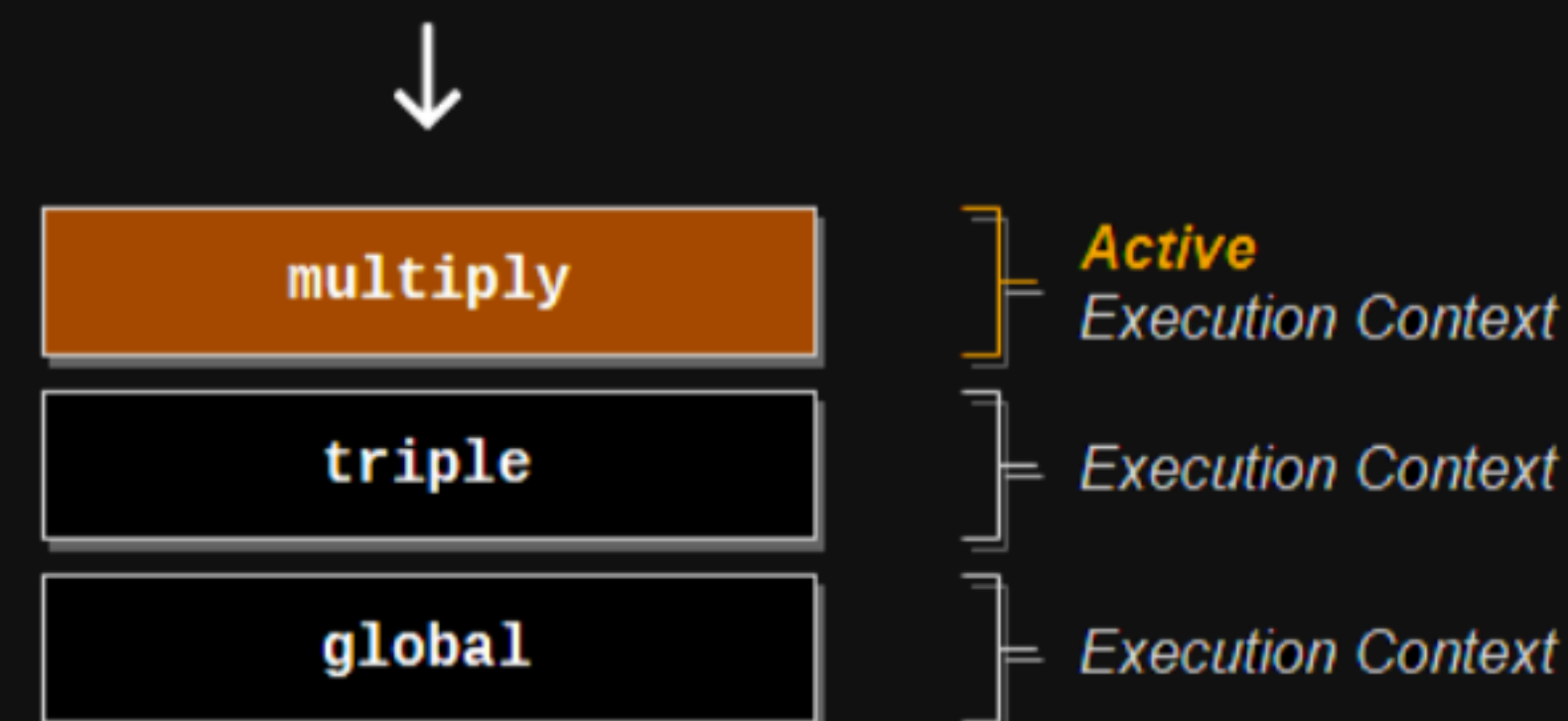
- Heap is a large region of memory where objects are allocated
- Call Stack forms a stack of frames as functions are invoked

Call Stack



All JavaScript code runs within the scope of what is known as an Execution Context

```
// global execution context  
  
var FACTOR_OF_THREE = 3;  
  
function multiply(a, b) { // execution context (multiply)  
    return a * b;  
}  
  
function triple(a) { // execution context (triple)  
    return multiply(a, FACTOR_OF_THREE);  
}  
  
// Invoking the same function twice generates four different  
// execution contexts, two on each function call  
triple(10);  
triple(6);
```



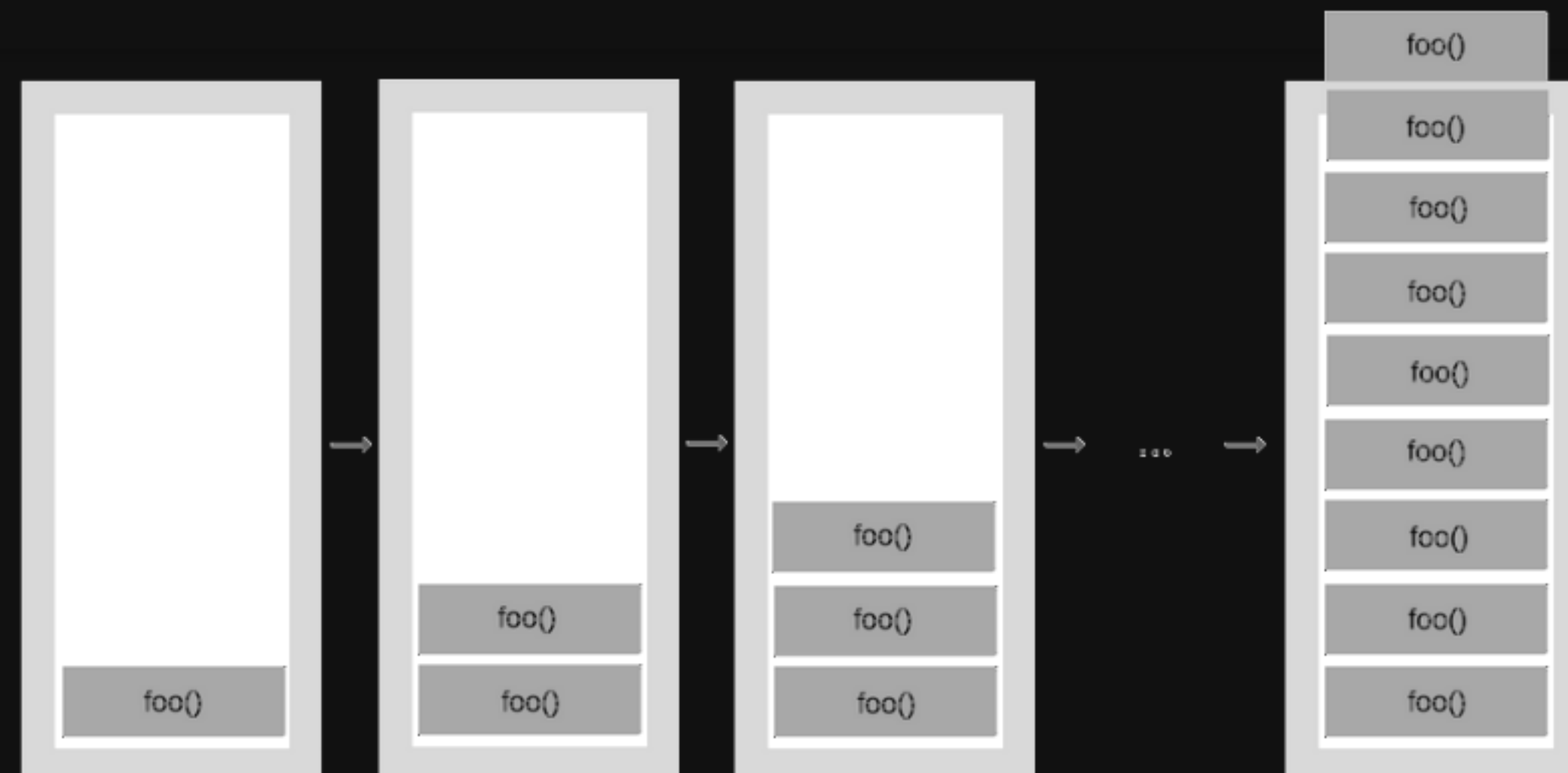
- A global execution context is created initially and pushed into the call stack
- Every time a function is invoked a new execution context is created and pushed into the call stack
- Every time a function returns its execution context is popped from the call stack

Stack Overflow



If the maximum call stack size is exceeded the program terminates with a *RangeError*

```
function foo() {  
  foo(); // call foo recursively forever  
}  
  
foo();
```



Code Execution



While executing code, the runtime engine goes through two different phases multiple times, until there is no more code to run

1. **Creation Phase** - A new Execution Context gets created
2. **Execution Phase** - During this phase, the following actions occur:
 - Created EC is stored in the call stack
 - Code from the EC is executed
 - EC is removed from the call stack
 - Code from the previous EC continues to execute

Execution Context



It is possible to conceptually represent each execution context (EC) as an object with three properties

- Activation Object stores all variables defined within the current EC
- Scope Chain gives access to the Activation Object for all parent lexical scopes
- This Binding is a reference to the object that owns the currently executing code

Execution context	
Variable object	{ vars, function declarations, arguments... }
Scope chain	[Variable object + all parent scopes]
thisValue	Context object

Activation Object



When a function is activated (called), the interpreter scans the function for arguments and local variables or local function declarations

```
function foo(x, y) {  
    var z = 30;  
    function bar() {}  
}  
foo(10, 20);
```

Activation object	
x	10
y	20
arguments	{0: 10, 1: 20, ..}
z	30
bar	<function>

The result of this scan becomes the Activation object and is the reason why hoisting occurs

Scope Chain



Container of Activation Objects for the executing function lexical scope

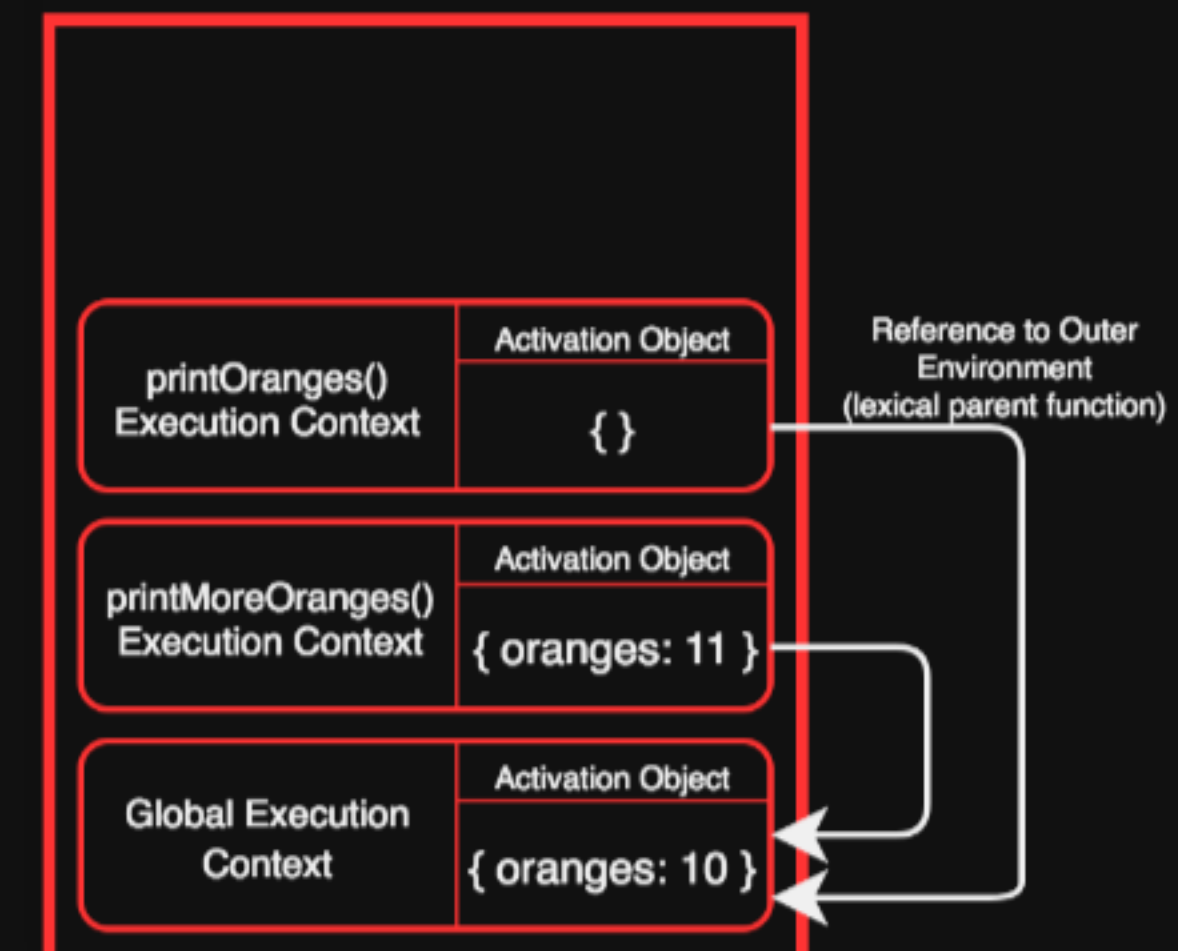
```
var oranges = 10;

function printOranges() {
  console.log(oranges);
}

function printMoreOranges() {
  var oranges = 11;
  printOranges();
}

printMoreOranges(); // what is the output?
```

Call Stack



this Argument Binding

The implicit this argument is available inside functions



```
function getOwner() {  
    console.log(this.owner);  
}  
  
var owner = "global";  
  
var obj1 = {  
    owner: "obj1",  
    getOwner: getOwner  
};  
  
var obj2 = {  
    owner: "obj2"  
};  
  
getOwner();           // "global"  
obj1.getOwner();      // "obj1"  
getOwner.call( obj2 ); // "obj2"  
new getOwner();        // undefined
```

The value of **this** is dynamically bound depending on how the function was called!

this Pitfalls



The implicit **this** binding can produce strange results

```
// get a collection of all button elements on the page
var buttons = document.getElementsByTagName('button');

for (var i = 0; i < buttons.length; i++) {
  buttons[i].onclick = function() {
    // log the button pressed to the console
    console.log(this.innerText + ' was pressed!');

    // make sure every button on page is only pressed once
    disableButton();
  }
}

function disableButton() {
  this.disabled = true;
}
```

How do you fix this?

Altering **this** binding



The methods bind, call and apply allow explicit control over the **this** argument binding

```
function compose(prefix, suffix) {  
    console.log(prefix + this + suffix);  
}  
  
// create a new function with a statically set this argument  
var compose2 = compose.bind('Rui');  
compose2('Hello ', '!');  
  
// call an existing function with an explicitly provided this argument  
compose.call('Rui', 'Hello ', '!'); // passing arguments as usual  
compose.apply('Rui', ['Hello ', '!']); // passing arguments as an array
```

Borrowing Methods



JavaScript makes it possible to use one object's method on a totally different object

The arguments object looks like an array with numerical indexed keys, but lacks all of the array methods

```
// this blows up, the arguments object lacks the native array methods
function upperCaseArguments() {

    return arguments.forEach(function(arg) {
        return arg.toUpperCase();
    });
}

// this is really smart :)
function upperCaseArguments() {

    // invoke the array slice method on top of the arguments object
    var argsAsArray = [].slice.call(arguments);

    return argsAsArray.map(function(arg) {
        return arg.toUpperCase();
    });
}
```

Closures

Closures



Accessing variables up the scope chain will make the interpreter preserve the corresponding EC and create what is called a closure

Closures effectively enable JavaScript asynchronous programming style

```
addImgClickHandler();

function addImgClickHandler() {
  var counter = 0;
  var myImage = document.querySelector('img');

  myImage.onclick = function() {
    counter++;
    console.log('You have clicked this image ' + counter + 'times');
  }
}
```

By the time the user clicks the button, the execution of addImgClickHandler has long terminated,
but the counter variable is still accessible

Explicit Closures



Closures seem complex at first, but they are a powerful tool with many practical use cases, allowing for creative, expressive and concise code

```
var twice = multiplier(2);
console.log(twice(5));

function multiplier(factor) {
  return function(number) {
    return number * factor;
  };
}
```

The AO created upon the execution of multiplier is preserved and available during the execution of twice

Module Pattern



Closures can be utilized to emulate the concept of classes with public and private members

```
var secretFactory = function(secret, key) {  
  var tries = 3;  
  
  return {  
    get: function(secretKey) {  
      tries -= 1;  
      return (tries > 0 && key === secretKey) ? secret : null;  
    }  
  };  
};  
  
var secret1 = secretFactory('coca-cola recipe', 1234);  
var secret2 = secretFactory('a message', 'password');  
  
console.log(secret1.get('1234')); // null  
console.log(secret1.get()); // null  
console.log(secret1.get('password')); // null  
console.log(secret1.get(1234)); // null, secret has been destroyed  
  
console.log(secret2.get('password')); // a message
```

Closure Pitfalls



Closures together with lexical scope can produce strange results

```
var funcs = [];  
for (var i = 0; i < 3; i++) {  
    // push will create new execution context with reference to i  
    funcs.push(function() {  
        console.log(i);  
    });  
}  
  
funcs[0](); // 3  
funcs[1](); // 3  
funcs[2](); // 3
```

How do you fix this?



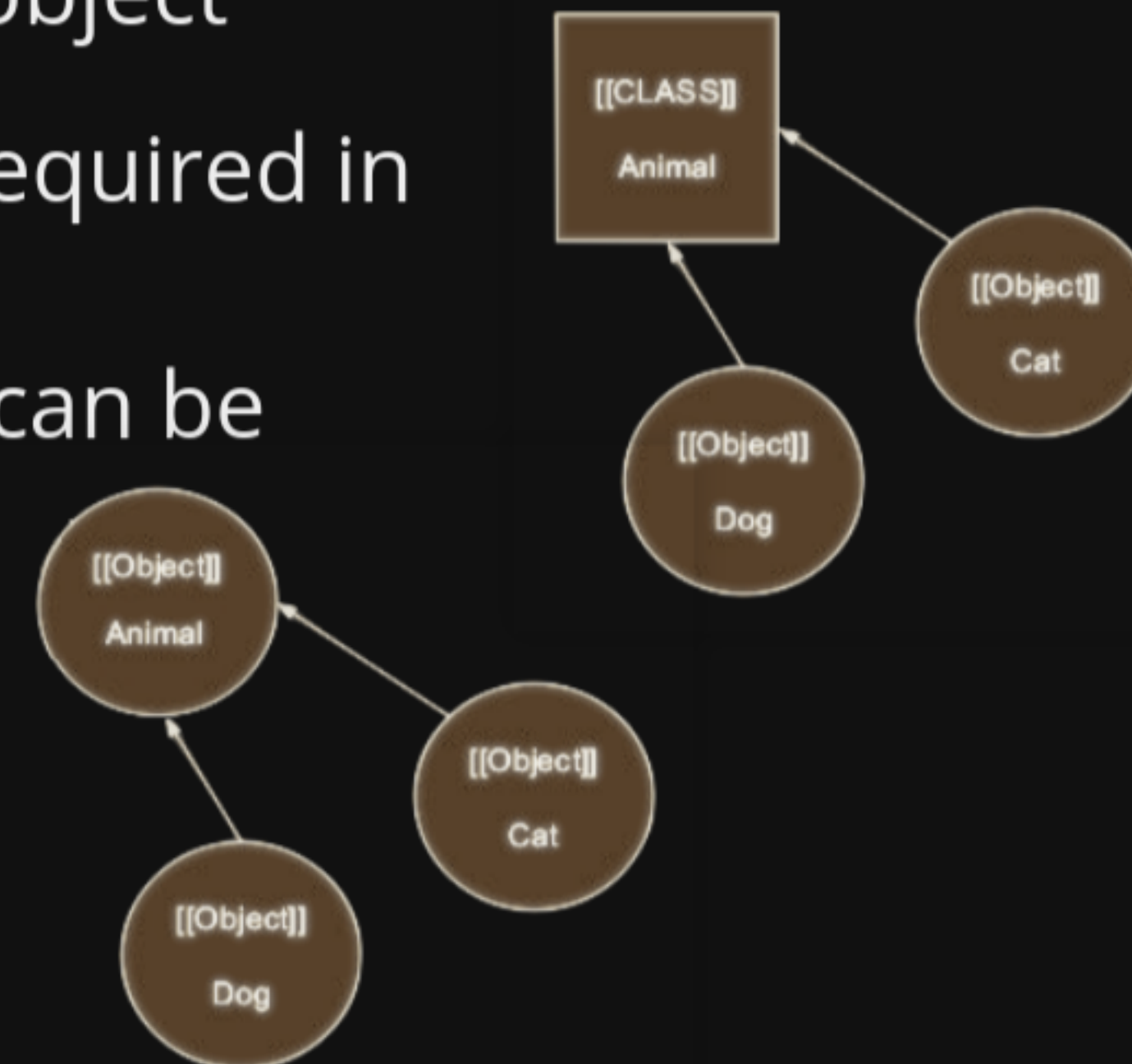
Object Creation and Inheritance

Inheritance

Through inheritance an object gets access to properties and methods of another object

- Classical Inheritance - a class blueprint is required in order to create an object
- Prototypal Inheritance - an existing object can be used as a prototype for other objects

JavaScript prototypal inheritance is so flexible that it can emulate classical inheritance!

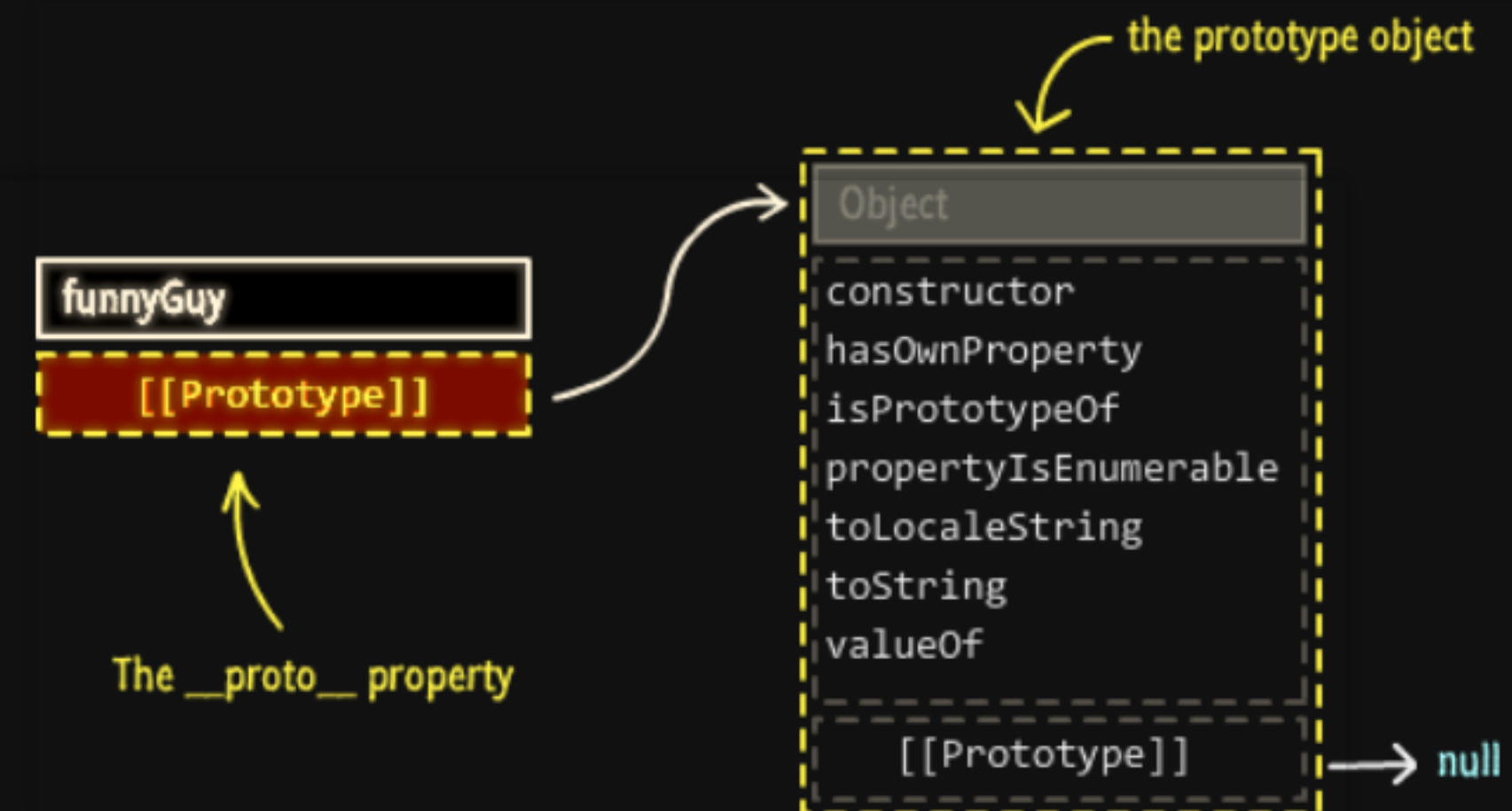


Prototypal Inheritance



Based on a delegation mechanism in which non existent properties are looked up in prototype objects

```
var funnyGuy = {  
  firstName: 'Pedro',  
  lastName: 'Antoninho'  
};  
  
console.log(funnyGuy);  
console.log(funnyGuy.valueOf());  
console.log(funnyGuy.__proto__.valueOf.call(funnyGuy));
```



All JavaScript objects delegate to an object referenced by Object.prototype!

the __proto__ property should not be manipulated directly

Prototype Link



It is possible to create a new object that delegates to an existing one

```
var foo = {  
  a: 42  
};  
  
// create bar and link it to foo  
var bar = Object.create(foo); // bar.__proto__ = foo  
bar.b = 'hello world';  
  
console.log(bar.b);      // 'hello world'  
console.log(bar.a);      // 42 <-- delegated to foo  
  
// shadowing foo.a  
bar.a = 2;               // hides foo.a  
console.log(bar.a);      // 2  
console.log(foo.a);      // 42
```

Shadowing occurs because properties are only looked up the prototype chain if not present on the object

Constructor Functions



A constructor function is used to create multiple instances of the same type of objects, all delegating to a common prototype

```
function Shape(color, borderThickness) {  
  this.color = color;  
  this.borderThickness = borderThickness;  
}  
  
var shape = new Shape('red', 2.0);
```

```
// roughly the equivalent of performing  
// var shape = new Shape('red', 2.0);  
  
var shape = Object.create(Shape.prototype);  
Shape.call(shape, 'red', 2.0);
```

They emulate classes, should only be used with the new operator and should be capitalized for readability

All functions come with a prototype property that:

- references an object that delegates to Object.prototype
- contains a constructor property that references the function itself
- can be used to expose methods to all objects created from the constructor function

DO NOT confuse the prototype property of a function with the prototype of an object (__proto__)!

Pseudo-Classical Inheritance



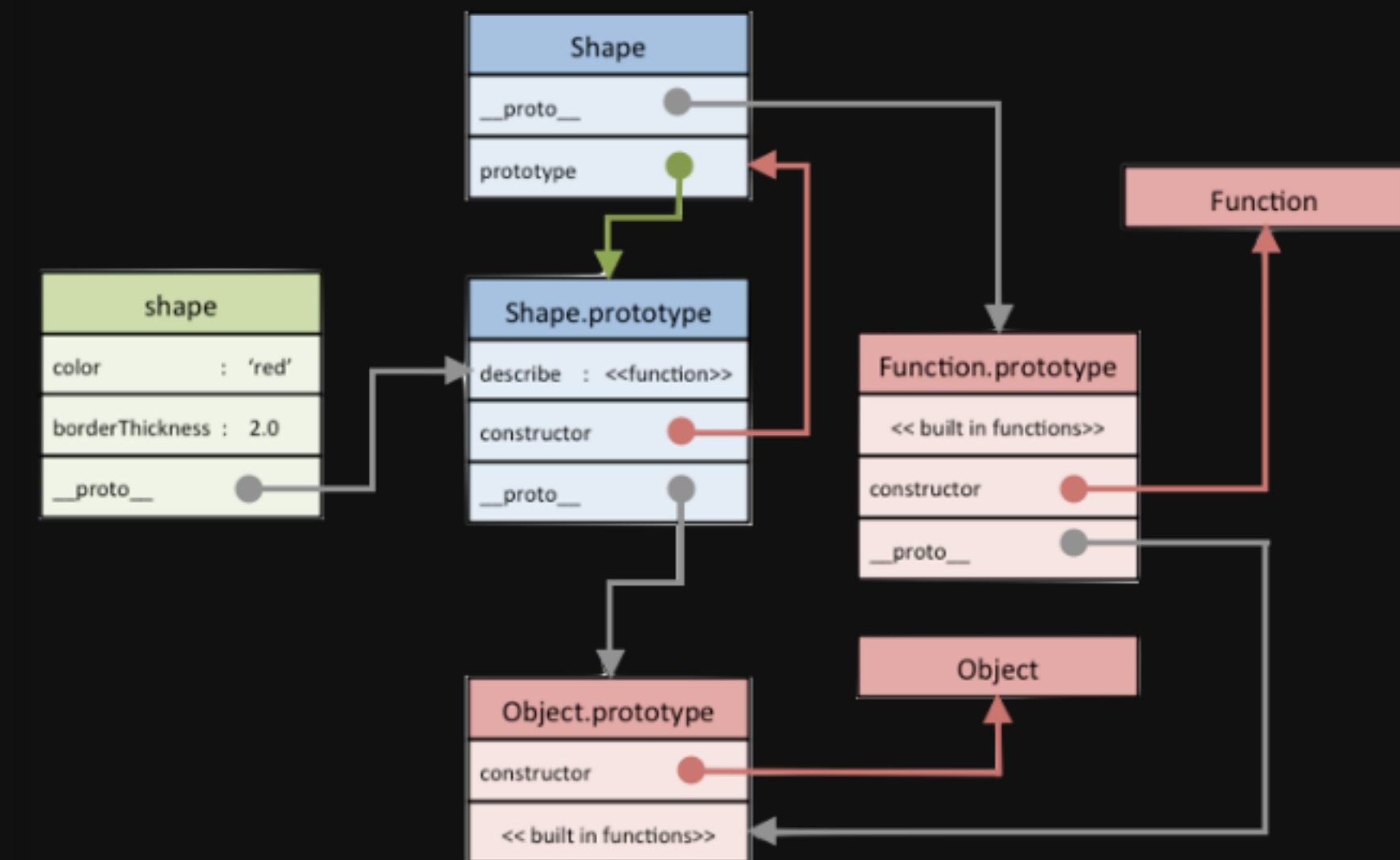
Despite its prototypal nature, classical inheritance can be emulated in Javascript with constructor functions and the new operator

```
// Constructor function for objects of type Shape
function Shape(color, borderThickness) {
  // property initialization code inside constructor function
  this.color = color;
  this.borderThickness = borderThickness;
}

// methods inside the prototype property of constructor function
Shape.prototype.show = function() {
  console.log(this.borderThickness + ' ' + this.color + ' shape ');
}

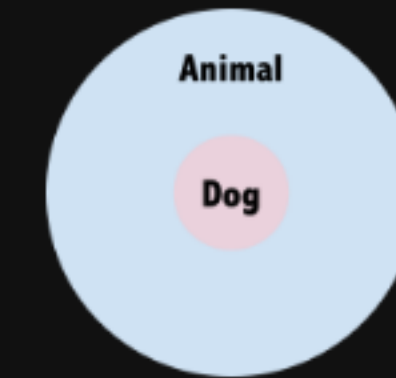
var shape = new Shape('red', 2.0); // create a new Shape instance
shape.show(); // 2 red shape
console.log(typeof shape); // object

// check if Shape.prototype is in shape prototype chain
console.log(shape instanceof Shape); // true
Shape.prototype = {}; // set default Shape prototype to an empty object
console.log(shape instanceof Shape); // false
```



Subclassing

Subclassing can be achieved in pseudo-classical inheritance by chaining and linking constructors together



```
// Animal Constructor
function Animal(name) {
    this.name = name;
}

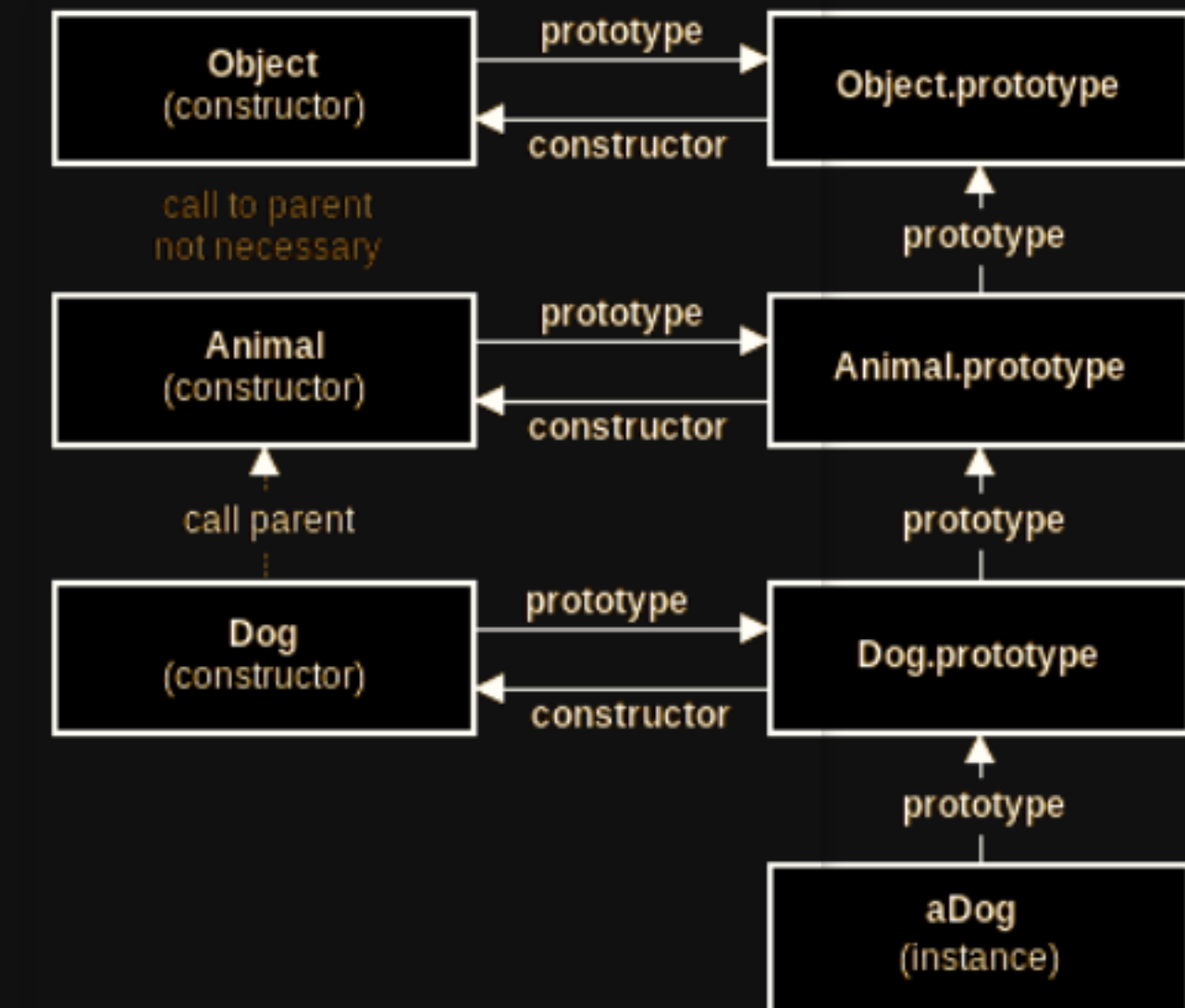
// Animal Methods
Animal.prototype.walk = function() {
    console.log(this.name + ' is walking');
}

// Dog Constructor
function Dog(name, breed) {
    // chain Animal and Dog constructors
    Animal.call(this, name);
    this.breed = breed;
}

// Make Dog inherit from Animal by overriding Dog prototype object
// with a new empty object that delegates to Animal.prototype
Dog.prototype = Object.create(Animal.prototype);

// Recreate constructor property destroyed in previous line
Dog.prototype.constructor = Dog;

// Dog Methods
Dog.prototype.bark = function() {
    console.log(this.name +
        ' is from breed' + this.breed +
        ' and says rauf rauf');
}
```



Error Handling



Errors



When the JS engine encounters an erroneous situation, its normal flow is interrupted by doing what is known as throwing an error:

- Early Errors - thrown during parsing, can not be handled
- Runtime Errors - thrown during execution, can be handled by **catching the error**

```
var x = 2; // this line will never be executed
console.log(x);

// this line will trigger an early syntax error
console.log(x + 1;
```

Syntax errors normally occur early, but sometimes they can occur at runtime as well

```
var x = 2;
console.log(x); // prints the value 2

// this line will trigger a runtime syntax error
var y = new Function('console.log(x + 1;');
```

Error Handling



It is possible to mark a block of statements in the code and specify a response if an error is thrown at runtime

```
try {  
    var myObject = {};  
    myObject.method(); // throws an error  
} catch (err) {  
    console.log('An error has occurred: ', err);  
} finally {  
    console.log('Error has been handled!');  
}
```

Error Objects



JavaScript error handling is typically performed through the generic **Error** constructor or one of its subconstructors

```
try {  
    var myObject = {};  
    myObject.method(); // throws an error  
} catch (err) {  
    console.log(typeof err); // object  
    console.log(err instanceof Error); // true  
    console.log(err instanceof TypeError); // true  
  
    // print a human readable description of the error  
    console.log(err.name + ': ' + err.message); // TypeError: myObject.method is not a function  
  
    if (debug) {  
        // dump the full stack trace to the console  
        console.log(err.stack);  
    }  
}
```

Error Throwing



An error can be thrown to signal a runtime error in the execution of the code

The execution of the current function will stop and control passed to the first *catch* clause in the call stack

```
function doSomethingRisky() {  
    throw new Error('something went wrong...');  
}  
  
try {  
    doSomethingRisky();  
} catch (err) {  
    console.log(err.name); // Error  
    console.log(err.message); // something went wrong  
}
```

If no *catch* clause exists, the program will be terminated

You can throw anything, but it is a good idea to throw Error objects only

Custom Errors



The Error object can also be used as a base object for custom errors

```
function CustomError(message, module) {  
    // Error is not your typical constructor function  
    var error = Error(message); // chain CustomError and Error constructors  
    this.name = 'CustomError'; // the standard error name property  
    this.message = error.message; // the standard message property  
    this.stack = error.stack; // the stack trace  
  
    this.module = module; // the custom module property  
}  
  
// Make CustomError inherit from a new error object,  
CustomError.prototype = Object.create(Error.prototype);  
CustomError.prototype.constructor = CustomError;  
  
try {  
    throw new CustomError('wrong credentials', 'authentication');  
} catch(e){  
    console.log(e.name + ': ' + e.module + ' - ' + e.message);  
    console.log(e.stack);  
}
```

Error does not require new and does not manipulate *this*, it simply returns a new Error instance

{ Exercise }

✓ you should get a green check mark on all tests

```
1. ferrao@Ruis-MacBook-Pro-3: ~/Dev/Workspace/academia-codigo/js-bootcamp/exercises/03-jsdeep/js-tests (zsh)

arrays
✓ you should be able to determine the location of an item in an array
✓ you should be able to sum the items of an array
✓ you should be able to remove all instances of an item from an array
✓ you should be able to remove all instances of an item from an array, returning the original array
✓ you should be able to add an item to the end of an array
✓ you should be able to remove the last item of an array
✓ you should be able to add an item to the beginning of an array
✓ you should be able to remove the first item of an array
✓ you should be able to join together two arrays
✓ you should be able to add an item anywhere in an array
✓ you should be able to count the occurrences of an item in an array
✓ you should be able to find duplicates in an array
✓ you should be able to square each number in an array
✓ you should be able to find all occurrences of an item in an array

strings
✓ you should be able to reduce duplicate characters to a desired minimum
✓ you should be able to wrap lines at a given number of columns, without breaking words
✓ you should be able to reverse a string
✓ you should be able to check if string is palindrome

51 passing (88ms)

~/Dev/Workspace/academia-codigo/js-bootcamp/exercises/03-jsdeep/js-tests master →
```